ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

		Скобцов Ю.А.
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 5

Оптимизация многомерных функций с помощью эволюционной стратегии
Вариант 4
по курсу: Эволюционные методы проектирования программно-информационных систем

РАБОТУ ВЫПОЛНИЛ
СТУДЕНТ

гр. № 4332К		Могилатов С.И.
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 УСЛОВИЕ ЗАДАНИЯ 3

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 4

2.1 Отличия эволюционной стратегии 4

2.2 Многократная ES 4

2.3 Самоадаптация параметров 4

3 ПРОГРАММА И РЕЗУЛЬТАТЫ 5

3.1 Алгоритм и параметры реализации 5

3.2 Результаты для n=2 и n=3 5

3.3 Исследование параметров 7

4 ВЫВОДЫ 9

5 ОТВЕТ НА КОНТРОЛЬНЫЙ ВОПРОС 10

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 11

Приложение А. Листинг эволюционной стратегии 12

Приложение Б. Сценарий экспериментов 14

1 УСЛОВИЕ ЗАДАНИЯ

Для варианта 4 требуется реализовать эволюционную стратегию для оптимизации многомерной функции moved axis parallel hyper-ellipsoid. В методических указаниях формула приведена как f(x) = sum(5*i*x_i^2), i=1..n, а в графе оптимума указано x_i=5*i. Эти два фрагмента противоречат друг другу. В работе формула используется буквально, поэтому математически согласованный глобальный минимум равен f(x*)=0 в точке x*=(0,...,0).

Требуется выполнить поиск для n=2, построить график функции с точками популяции и найденным экстремумом, исследовать влияние размера популяции и вероятности мутации, а затем повторить эксперимент для n=3 и сравнить результаты по точности и времени.

Таблица 1 - Параметры варианта 4

Показатель	**Значение**
Функция	sum(5*i*x_i^2), i=1..n
Область	-5,12 <= x_i <= 5,12
Теоретический минимум	0
Точка минимума по формуле	x_i=0
Базовая популяция	mu=lambda=20
Критерий останова	100 поколений или стагнация 25 поколений
Контрольный вопрос	4

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Отличия эволюционной стратегии

Эволюционная стратегия выполняет поиск в пространстве вещественных фенотипов. Потенциальное решение задается вектором координат x, а стратегические параметры определяют величину случайного шага. В отличие от двоичного генетического алгоритма не требуется кодировать и декодировать хромосому.

2.2 Многократная ES

В работе использована стратегия (mu+lambda). Из mu родителей формируется lambda потомков, после чего родители и потомки объединяются, а mu лучших особей переходят в следующее поколение. В базовой серии mu=lambda=20. Мутация выбирает родителя, изменяет его координаты нормальным шумом и ограничивает результат областью допустимых значений.

2.3 Самоадаптация параметров

Вместе с координатами изменяется вектор sigma стандартных отклонений. Новое значение sigma получается логнормальным масштабированием с общим и индивидуальным случайными множителями. Большие значения sigma обеспечивают исследование области, малые позволяют уточнять решение около найденного минимума.

3 ПРОГРАММА И РЕЗУЛЬТАТЫ

3.1 Алгоритм и параметры реализации

Файл evolution_strategy.py содержит функцию целевого значения, класс Individual, мутацию, самоадаптацию sigma, отбор (mu+lambda) и критерий остановки. Сценарий run_experiments.py запускает две базовые серии, исследует параметры и формирует изображения. Для каждого поколения сохраняются лучшее значение, среднее значение популяции, расстояние до нулевой точки и среднее sigma.

Таблица 2 - Параметры базовой серии

Параметр	Значение
Размер популяции mu	20
Число потомков lambda	20
Вероятность мутации	1,00
Границы координат	[-5,12; 5,12]
Максимум поколений	100
Лимит стагнации	25
Seed	6 для n=2, 7 для n=3

3.2 Результаты для n=2 и n=3

Таблица 3 - Сравнение размерности задачи

Размерность	Лучшее f(x)	Точка решения	Поколение остановки	Время, с
n=2	1.1225e-13	['-8.463e-08', '8.743e-08']	80	0.005426
n=3	3.6199e-10	['-3.868e-06', '5.227e-06', '-9.643e-07']	100	0.009219

Для n=2 получено f(x)=1.122538e-13, координаты имеют порядок 10^-7. Для n=3 получено f(x)=3.619923e-10, координаты имеют порядок 10^-6. Увеличение размерности потребовало больше поколений и немного увеличило время вычисления, но найденная точка также близка к теоретическому минимуму.

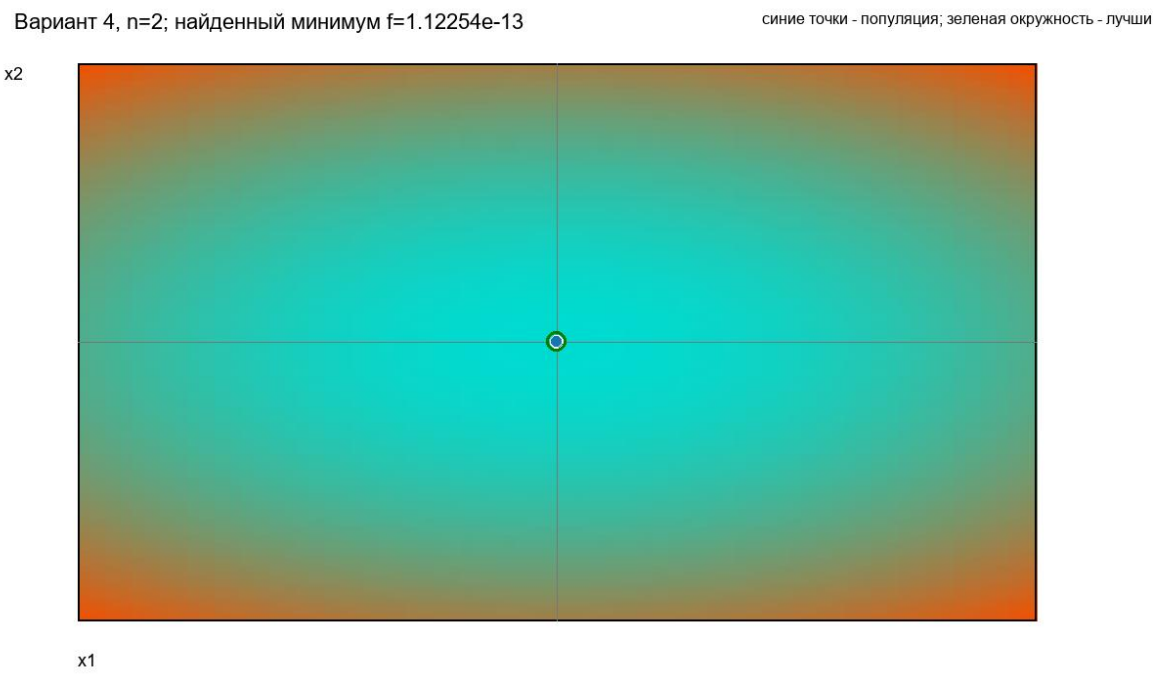


Рисунок 1 - Функция варианта 4, точки популяции и найденный экстремум

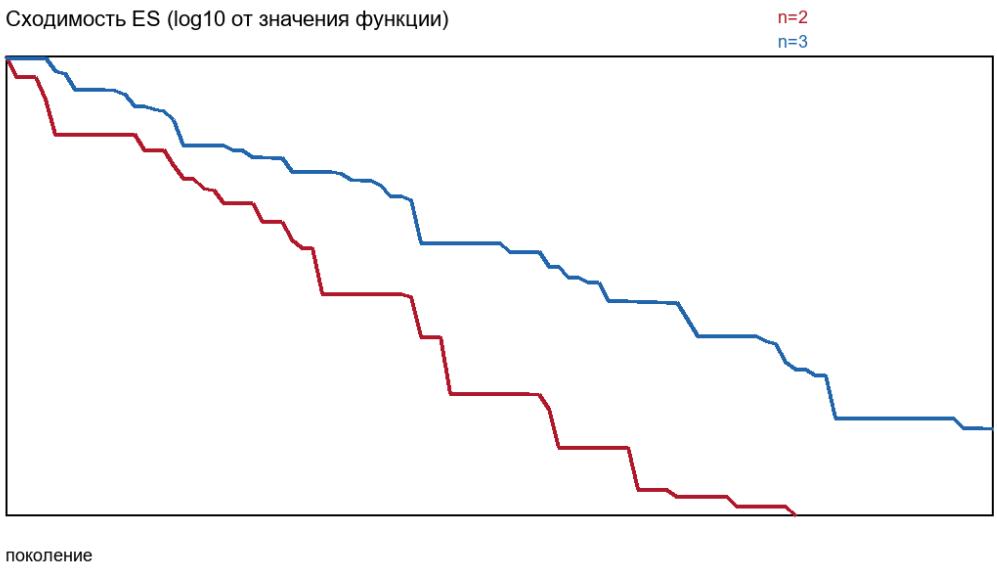


Рисунок 2 - Сходимость ES для n=2 и n=3

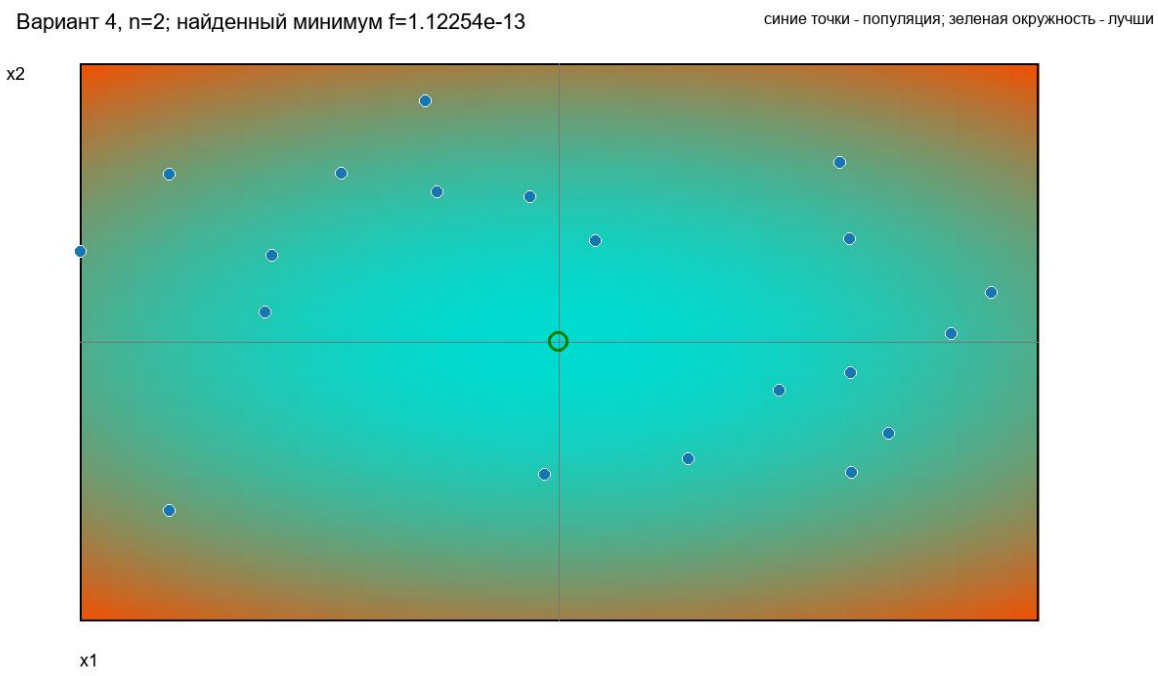


Рисунок 3 - Начальная популяция для n=2

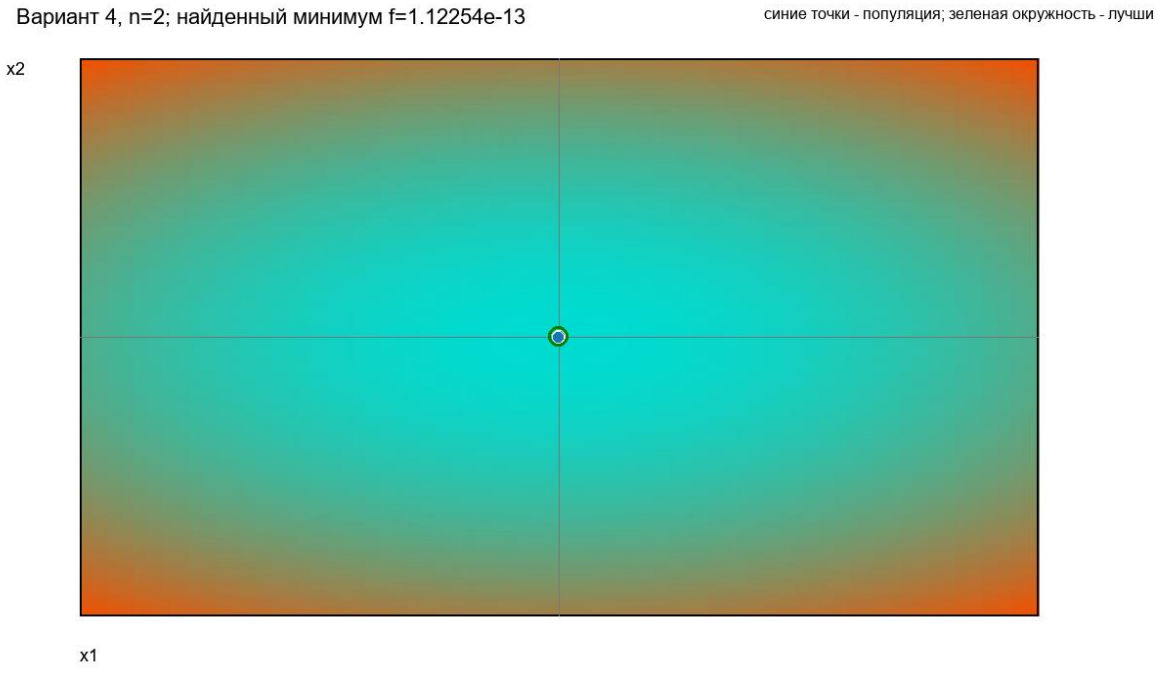


Рисунок 4 - Промежуточная популяция для n=2

3.3 Исследование параметров

Исследованы размер популяции mu=5, 10, 20, 40 и вероятность мутации p=0,25, 0,50, 0,75, 1,00. Для остальных настроек использовались 100

поколений и одинаковый критерий стагнации. Качество сравнивалось по достигнутому значению целевой функции, а производительность - по времени и числу поколений.

Таблица 4 - Параметрическое исследование

Фактор	Значение	Лучшее f(x)	Поколения	Время, с	Расстояние
population	mu=5	2.491e-09	100	0.001903	1.829e-05
population	mu=10	9.975e-13	90	0.003449	3.539e-07
population	mu=20	9.219e-14	89	0.008614	1.257e-07
population	mu=40	2.425e-13	83	0.014152	2.189e-07
mutation_probability	p=0.25	3.497e-09	100	0.003846	1.998e-05
mutation_probability	p=0.50	2.427e-02	34	0.001879	4.980e-02
mutation_probability	p=0.75	1.171e-03	39	0.002007	1.351e-02
mutation_probability	p=1.00	3.623e-13	76	0.005779	1.920e-07

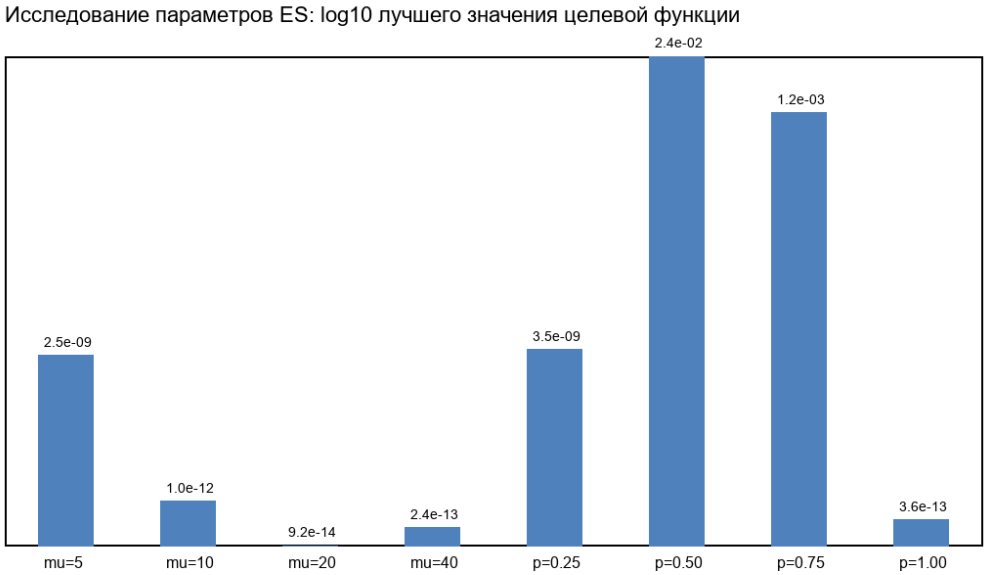


Рисунок 5 - Зависимость результата от параметров ES

Увеличение популяции в проведенных сериях улучшило вероятность достижения окрестности минимума, но увеличило время одного запуска. Вероятность мутации p=1,00 дала наиболее точное решение в базовом сравнении. При p=0,50 остановка по стагнации произошла раньше и результат оказался хуже, что показывает чувствительность ES к интенсивности исследования.

4 ВЫВОДЫ

Реализована вещественная эволюционная стратегия (mu+lambda) с нормальной мутацией и самоадаптацией стандартных отклонений. Для n=2 найдено значение функции 1,12*10^-13, для n=3 - 3,62*10^-10, что подтверждает приближение к математически согласованному минимуму печатной формулы.

Пошаговые графики показывают движение точек популяции к началу координат. Исследование параметров выявило компромисс: увеличение популяции повышает разнообразие поиска, но увеличивает время; слишком малая вероятность мутации может привести к преждевременной стагнации.

5 ОТВЕТ НА КОНТРОЛЬНЫЙ ВОПРОС

Двукратная эволюционная стратегия обозначается (1+1)-ЭС. В ней в каждый момент времени поддерживается одна родительская особь. Из нее мутацией получают одного потомка: координаты потомка равны координатам родителя плюс нормально распределенные случайные приращения, масштаб которых задается вектором sigma. Затем сравнивают значения целевой функции. Если потомок лучше родителя, он становится новой родительской особью; если нет, потомок отбрасывается. Процесс повторяется до достижения критерия остановки.

Преимуществом (1+1)-ЭС является простота и небольшое количество вычислений, недостатком - отсутствие разнообразия популяции и риск медленной сходимости или застревания. В данной работе для практического исследования числа особей использована многократная (mu+lambda)-ЭС, но ее мутационный принцип основан на той же идее.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лабораторная работа № 5. Оптимизация многомерных функций с помощью эволюционной стратегии: методические указания.

2. Вирсански Э. Генетические алгоритмы на Python / пер. с англ. - М.: ДМК Пресс, 2020.

3. Скобцов Ю. А. Вычислительный интеллект. - СПб.: ГУАП, 2022.

4. GUAP. Нормативная документация для учебного процесса. - URL: https://guap.ru/c/regdocs/docs/uch (дата обращения: 15.10.2026).

Приложение А. Листинг эволюционной стратегии

```
"""Evolution strategy for GUAP laboratory work 5, variant 4."""

from __future__ import annotations

import math
import random
from dataclasses import dataclass
from typing import Iterable



def objective(vector: Iterable[float]) -> float:
    """Printed variant-4 formula: sum(5*i*x_i^2), i starts at one."""
    return sum(5.0 * (index + 1) * value * value for index, value in enumerate(vector))



@dataclass
class Individual:
    x: list[float]
    sigma: list[float]
    value: float

    def clone(self) -> "Individual":
        return Individual(self.x[:], self.sigma[:], self.value)



@dataclass
class RunResult:
    dimension: int
    best: Individual
    history: list[dict]
    snapshots: list[list[Individual]]
    elapsed_seconds: float
    stopped_generation: int



class EvolutionStrategy:
    """Self-adaptive (mu+lambda)-ES with real-valued phenotype mutation."""

    def __init__(self, dimension: int, population_size: int = 20, offspring_count: int | None =
None,
                 mutation_probability: float = 1.0, bounds: tuple[float, float] = (-5.12, 5.12),
                 max_generations: int = 100, stagnation_limit: int = 25, seed: int = 4):
        self.dimension = dimension
        self.population_size = population_size
        self.offspring_count = offspring_count or population_size
        self.mutation_probability = mutation_probability
        self.lower, self.upper = bounds
        self.max_generations = max_generations
        self.stagnation_limit = stagnation_limit
        self.rng = random.Random(seed)
        self.tau = 1.0 / math.sqrt(2.0 * math.sqrt(dimension))
        self.tau_prime = 1.0 / math.sqrt(2.0 * dimension)

    def make_individual(self) -> Individual:
        x = [self.rng.uniform(self.lower, self.upper) for _ in range(self.dimension)]
        sigma = [(self.upper - self.lower) * 0.2 for _ in range(self.dimension)]
        return Individual(x, sigma, objective(x))

    def mutate(self, parent: Individual) -> Individual:
        if self.rng.random() > self.mutation_probability:
            return parent.clone()
```

```
        common = self.rng.gauss(0.0, 1.0)
        sigma = []
        x = []
        for value, step in zip(parent.x, parent.sigma):
            new_step = step * math.exp(self.tau_prime * common + self.tau * self.rng.gauss(0.0,
1.0))
            new_step = max(1e-8, min(self.upper - self.lower, new_step))
            new_value = value + new_step * self.rng.gauss(0.0, 1.0)
            x.append(max(self.lower, min(self.upper, new_value)))
            sigma.append(new_step)
        return Individual(x, sigma, objective(x))


    def run(self) -> RunResult:
        import time


        started = time.perf_counter()
        population = [self.make_individual() for _ in range(self.population_size)]
        best = min(population, key=lambda individual: individual.value).clone()
        history: list[dict] = []
        snapshots: list[list[Individual]] = []
        stagnant = 0
        for generation in range(self.max_generations + 1):
            population.sort(key=lambda individual: individual.value)
            if population[0].value < best.value:
                best = population[0].clone()
                stagnant = 0
            else:
                stagnant += 1
            history.append({
                "generation": generation,
                "best_value": best.value,
                "mean_value": sum(item.value for item in population) / len(population),
                "distance": math.sqrt(sum(value * value for value in best.x)),
                "sigma_mean": sum(best.sigma) / len(best.sigma),
            })
            if generation % 10 == 0 or generation == self.max_generations:
                snapshots.append([item.clone() for item in population])
            if  generation  ==  self.max_generations  or  stagnant  >=  self.stagnation_limit  or
best.value < 1e-12:
                break
            offspring     =     [self.mutate(self.rng.choice(population))     for     _     in
range(self.offspring_count)]
            population    =    sorted(population    +    offspring,    key=lambda    individual:
individual.value)[: self.population_size]
        return RunResult(self.dimension, best, history, snapshots, time.perf_counter() - started,
generation)
```

Приложение Б. Сценарий экспериментов

```
"""Run reproducible evolution-strategy experiments and create figures."""

from __future__ import annotations

import csv
import json
import math
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont

from evolution_strategy import EvolutionStrategy, Individual, objective



ROOT = Path(__file__).resolve().parent
RESULTS = ROOT / "results"
RESULTS.mkdir(exist_ok=True)
FONT_PATH = r"C:\Windows\Fonts\arial.ttf"



def font(size: int):
    return ImageFont.truetype(FONT_PATH, size)



def color(value: float, maximum: float) -> tuple[int, int, int]:
    ratio = max(0.0, min(1.0, value / max(maximum, 1e-12)))
    return (int(245 * ratio), int(80 + 140 * (1 - ratio)), int(210 * (1 - ratio)))



def map_xy(x: float, y: float, bounds: tuple[float, float], box: tuple[int, int, int, int]) ->
tuple[float, float]:
    left, top, right, bottom = box
    lo, hi = bounds
    return left + (x - lo) / (hi - lo) * (right - left), bottom - (y - lo) / (hi - lo) * (bottom
- top)



def plot_population(result, path: Path) -> None:
    """Draw a contour-like view for n=2 and the final population."""
    canvas = Image.new("RGB", (1200, 760), "white")
    draw = ImageDraw.Draw(canvas)
    box = (100, 80, 1080, 650)
    lo, hi = -5.12, 5.12
    grid = 70
    max_value = objective([hi, hi])
    for row in range(grid):
        for col in range(grid):
            x = lo + (hi - lo) * (col + 0.5) / grid
            y = lo + (hi - lo) * (row + 0.5) / grid
            px = int(box[0] + col * (box[2] - box[0]) / grid)
            py = int(box[1] + (grid - 1 - row) * (box[3] - box[1]) / grid)
            cw = int((box[2] - box[0]) / grid) + 1
            ch = int((box[3] - box[1]) / grid) + 1
            draw.rectangle((px, py, px + cw, py + ch), fill=color(objective([x, y]), max_value))
    draw.rectangle(box, outline="black", width=2)
    draw.line((box[0], (box[1] + box[3]) / 2, box[2], (box[1] + box[3]) / 2), fill="#777777",
width=1)
    draw.line(((box[0] + box[2]) / 2, box[1], (box[0] + box[2]) / 2, box[3]), fill="#777777",
width=1)
    final_population = result.snapshots[-1]
    for item in final_population:
```

```
        px, py = map_xy(item.x[0], item.x[1], (lo, hi), box)
        draw.ellipse((px - 6, py - 6, px + 6, py + 6), fill="#1677b5", outline="white")
    best_x, best_y = map_xy(result.best.x[0], result.best.x[1], (lo, hi), box)
    draw.ellipse((best_x - 10, best_y - 10, best_x + 10, best_y + 10), outline="#008000",
width=3)
    draw.text((35, 25), f"Вариант 4, n=2; найденный минимум f={result.best.value:.6g}",
fill="black", font=font(22))
    draw.text((box[0], 680), "x1", fill="black", font=font(18))
    draw.text((25, box[1]), "x2", fill="black", font=font(18))
    draw.text((800, 25), "синие точки - популяция; зеленая окружность - лучший результат",
fill="black", font=font(16))
    canvas.save(path)



def plot_convergence(results: dict[int, object], path: Path) -> None:
    canvas = Image.new("RGB", (1200, 650), "white")
    draw = ImageDraw.Draw(canvas)
    left, top, right, bottom = 110, 70, 1120, 540
    draw.rectangle((left, top, right, bottom), outline="black", width=2)
    colors = {2: "#b2182b", 3: "#2166ac"}
    max_gen = max(max(item["generation"] for item in result.history) for result in
results.values())
    max_log = max(math.log10(max(item["best_value"] for item in result.history) + 1e-8) for
result in results.values())
    min_log = min(math.log10(max(item["best_value"], 1e-12)) for result in results.values() for
item in result.history)
    if max_log <= min_log:
        max_log = min_log + 1
    for n, result in results.items():
        points = []
        for item in result.history:
            px = left + (right - left) * item["generation"] / max(1, max_gen)
            py = bottom - (bottom - top) * (math.log10(max(item["best_value"], 1e-12)) - min_log)
/ (max_log - min_log)
            points.append((px, py))
        draw.line(points, fill=colors[n], width=4)
        draw.text((900, 20 + (n - 2) * 25), f"n={n}", fill=colors[n], font=font(18))
    draw.text((left, 20), "Сходимость ES (log10 от значения функции)", fill="black",
font=font(22))
    draw.text((left, 570), "поколение", fill="black", font=font(18))
    canvas.save(path)



def plot_parameter_study(rows: list[dict], path: Path) -> None:
    canvas = Image.new("RGB", (1200, 700), "white")
    draw = ImageDraw.Draw(canvas)
    left, top, right, bottom = 100, 80, 1100, 580
    draw.rectangle((left, top, right, bottom), outline="black", width=2)
    values = [max(float(row["best_value"]), 1e-14) for row in rows]
    logs = [math.log10(value) for value in values]
    low, high = min(logs), max(logs)
    if high - low < 1e-9:
        high = low + 1
    for i, row in enumerate(rows):
        x = left + (right-left) * (i + 0.5) / len(rows)
        log_value = math.log10(max(float(row["best_value"]), 1e-14))
        bar_top = bottom - (bottom-top) * (log_value-low)/(high-low)
        draw.rectangle((x-28, bar_top, x+28, bottom), fill="#4f81bd")
        draw.text((x-23, bottom+8), str(row["label"]).replace("_", "\n"), fill="black",
font=font(16))
        draw.text((x-23, bar_top-22), f"{float(row['best_value']):.1e}", fill="black",
font=font(14))
    draw.text((left, 25), "Исследование параметров ES: log10 лучшего значения целевой функции",
fill="black", font=font(22))
    canvas.save(path)



def main() -> None:
    base_results = {}
```

```
    for n in (2, 3):
        base_results[n] = EvolutionStrategy(n, population_size=20, mutation_probability=1.0,
max_generations=100, seed=4 + n).run()
    plot_population(base_results[2], RESULTS / "population_n2.png")
    plot_convergence(base_results, RESULTS / "convergence_n2_n3.png")
    for index, population in enumerate(base_results[2].snapshots):
        if index in {0, 5, 10}:
            snapshot_result = base_results[2]
            snapshot_result.snapshots = [population]
            plot_population(snapshot_result, RESULTS / f"step_n2_{index * 10:03d}.png")


    study_rows = []
    for population in (5, 10, 20, 40):
        result = EvolutionStrategy(2, population_size=population, mutation_probability=1.0,
max_generations=100, seed=10 + population).run()
        study_rows.append({"factor": "population", "label": f"mu={population}", "value":
population, "best_value": result.best.value, "generations": result.stopped_generation,
"time_seconds": result.elapsed_seconds, "distance": math.sqrt(sum(value * value for value in
result.best.x))})
    for probability in (0.25, 0.50, 0.75, 1.00):
        result = EvolutionStrategy(2, population_size=20, mutation_probability=probability,
max_generations=100, seed=30 + int(probability * 100)).run()
        study_rows.append({"factor": "mutation_probability", "label": f"p={probability:.2f}",
"value": probability, "best_value": result.best.value, "generations": result.stopped_generation,
"time_seconds": result.elapsed_seconds, "distance": math.sqrt(sum(value * value for value in
result.best.x))})
    with (RESULTS / "parameter_study.csv").open("w", encoding="utf-8", newline="") as stream:
        writer = csv.DictWriter(stream, fieldnames=list(study_rows[0]))
        writer.writeheader(); writer.writerows(study_rows)
    plot_parameter_study(study_rows, RESULTS / "parameter_study.png")
    summary = {
        "variant": 4,
        "formula_printed": "f(x) = sum(5*i*x_i^2), i=1..n",
        "interpretation": "The printed formula is used literally; its mathematical minimum is
x_i=0. The nearby table entry x_i=5*i is recorded as an inconsistency in the source guide.",
        "bounds": [-5.12, 5.12],
        "population_size": 20,
        "offspring_count": 20,
        "mutation_probability": 1.0,
        "max_generations": 100,
        "stagnation_limit": 25,
        "base": {
            str(n): {"best_value": result.best.value, "best_x": result.best.x, "distance":
math.sqrt(sum(value * value for value in result.best.x)), "generations":
result.stopped_generation, "time_seconds": result.elapsed_seconds, "history": result.history}
            for n, result in base_results.items()
        },
        "study": study_rows,
    }
    (RESULTS / "results_summary.json").write_text(json.dumps(summary, ensure_ascii=False,
indent=2), encoding="utf-8")
    print(json.dumps({"n2": summary["base"]["2"], "n3": summary["base"]["3"]},
ensure_ascii=False, indent=2))



if __name__ == "__main__":
    main()
```